Automated ICD-10 Chapter Codification of Short Clinical Literals Using a TF-IDF and Clinical Transformer Blend

Hermes Barreiro Pena, Daniel Massoud, and Shanthosh

Abstract—This project addresses the automatic assignment of an ICD-10 chapter to short Spanish and Catalan medical literals, a fine-grained text classification task evaluated on a Kaggle leaderboard. The target, y category, corresponds to the first character of the ICD-10 code. The work proceeded in two phases. In the first, we built and systematically compared a wide range of classical approaches: exact and fuzzy matching, several retrieval methods, supervised linear classifiers, feature engineering, rule-based context overrides, and ICD-9-to-ICD-10 mapping. The strongest classical configuration, a combined character and word TF-IDF Linear SVM with selective context rules, reached a public score of about 0.572. In the second phase, we added a fine-tuned Spanish biomedical-clinical RoBERTa transformer and blended its probabilities with the classical ensemble, tuning the blend weight directly on the leaderboard. This improved the score to 0.595. An error analysis then identified the diagnosis-versus-procedure ambiguity of short literals as the dominant remaining source of error, characterising a practical ceiling for the task.

Keywords—*ICD-10 coding, clinical NLP, TF-IDF, information retrieval, transformer fine-tuning, model ensembling, multilingual text*

Contents

1. Introduction

This project focused on predicting the broad ICD category for short clinical literals. Each input was a short phrase: an abbreviation, a diagnosis, a procedure, or a context-based medical note. The required output was the y category, which generally corresponds to the first character of the ICD-10 code. Because the leaderboard scores a strict comparison on that first character, the problem reduces to a multi-class classification over the chapter labels rather than full code prediction.

The task was harder than a standard text classification problem because the literals were extremely short, noisy, and often ambiguous. Some examples contained Spanish or Catalan medical wording, spelling variations, abbreviations, ICD-like numeric strings, pregnancy-related context, or procedure names. Our motivation was to apply the techniques from the course to this realistic clinical dataset, compare several modelling strategies, and push the leaderboard score as far as the available text allowed.

2. Relation to Prior Work

Automatic ICD-10 coding of Spanish clinical text has been studied directly in the CodiEsp shared task (CLEF eHealth 2020), which released a corpus of clinical cases annotated with ICD-10-CM and ICD-10-PCS codes. Reported systems span the full spectrum: TF-IDF with linear classifiers as baselines, fine-tuned domain-specific BERT variants such as BERT-SciELO and BETO, and hybrid pipelines combining named-entity recognition, dictionary matching, and semantic linking.

Two findings from that literature shaped our second phase. First, domain-specific transformers consistently outperform general-purpose ones on clinical text, which guided our choice of a Spanish biomedical-clinical model over a generic multilingual one. Second, the addition of externally annotated corpora was reported to help, which motivated our use of external ICD-coded data as extra training material. Our contribution is not a new architecture but a careful, well-documented comparison of standard techniques, paired with an honest analysis of where the task saturates.

3. Data Preparation and Text Normalisation

3.1. Text Cleaning and Normalisation

The first step was to standardise the text into a cleaned literal column. This involved lowercasing, removing accents, normalising punctuation, removing unwanted characters, and collapsing repeated spaces. This was necessary because literals such as Retinopatía diabética 2, RETINOPATIA DIABETICA 2, and retinopatia diabetica 2 should be treated as the same phrase.

3.2. ICD-10 Category Creation

For the ICD-10 rows, the target category was created by taking the first character of the cleaned ICD-10 code. This works because ICD-10 categories are structured alphabetically or numerically in a way that matches the competition target. These rows became the main supervised training data.

3.3. ICD-9 Flag Identification

The dataset also included flagged or outdated ICD-9-style rows. These could not be used directly because the first digit of an ICD-9 code does not correspond to the ICD-10 category label; for example, an ICD-9 code beginning with 0 does not imply an ICD-10 category of 0. These rows were set aside for the later mapping experiments described in Section 6.

4. Phase 1: Classical Retrieval and Supervised Models

4.1. Matching and Retrieval-Based Methods

We began with matching and retrieval, the most direct approaches.

Exact matching compared normalised test literals directly against training literals. It worked when an identical literal appeared in training with a consistent label, but failed on unseen cases, spelling variants, and repeated literals with conflicting labels. It was useful as a diagnostic but too limited as the main method.

Fuzzy matching attempted to handle spelling mistakes by matching to the closest training literal. For abbreviations such as rn, dm, ev, or paf, however, small string differences were not reliable clinical evidence, so it often introduced noise.

Embedding / semantic retrieval compared literals by meaning rather than spelling. It underperformed because general embeddings

did not represent the many abbreviations, fragments, and code-like strings in the dataset well enough.

ICD description retrieval compared each literal against the official ICD descriptions. This was weak because official descriptions are formal and detailed while the literals are informal shorthand; the mismatch in style made it unreliable.

Literal-to-literal retrieval compared test literals against training literals instead, which matched the writing styles better, but still failed when similar literals belonged to different categories.

We then tested several voting variants. KNN retrieval predicted from the nearest TF-IDF neighbours; top-k voting took a majority over several neighbours for more stability; and weighted top-k voting gave more influence to closer neighbours. Each improved robustness slightly but still failed when neighbours were split across categories or were textually similar yet clinically different.

Centroid-based retrieval represented each category by an average vector, but performed worse because broad categories such as O, Z, C, or 0 are internally diverse and cannot be captured by a single centroid. BM25 retrieval added no clear advantage over TF-IDF. Finally, a retrieval-plus-SVC hybrid used retrieval only as a high-confidence fallback, but strict thresholds changed too few rows while relaxed thresholds introduced noisy overrides.

4.2. Supervised Machine Learning Models

We then moved to supervised classification. Logistic Regression on TF-IDF features was a reasonable baseline but weaker than a margin-based classifier, suggesting a Linear SVM was better suited to the high-dimensional sparse feature space.

We first attempted full ICD code prediction, but the long tail of rare full-code labels made this too difficult, so it was abandoned in favour of direct category prediction, which aligned with the evaluation format and used a smaller, more tractable label set.

Character TF-IDF with a Linear SVM became a strong early model, because character n-grams capture spelling variations, partial words, prefixes, suffixes, and abbreviations. The best core model combined character and word n-grams via a feature union: character features captured morphology while word features captured useful phrases such as ca mama, insuficiencia cardiaca, and parto.

A key finding was that this character-plus-word Linear SVM absorbed many of the cases that exact and fuzzy matching were designed for. It could generalise from similar forms without a separate fuzzy step, which explains why adding fuzzy matching afterwards did not help and sometimes added noise.

4.3. Tuning and Alternatives

The Linear SVM was tuned over its regularisation strength, character and word n-gram ranges, and minimum document frequency. The strongest stable Phase-1 configuration used character n-grams of length two to five, word n-grams of length one to two, a minimum document frequency of two, and a regularisation constant of approximately $C = 0.5$. (As described in Section 7, this value was re-tuned in the final blended system.)

Several alternatives were rejected. Class-weighted training overcorrected rare categories and reduced accuracy on common ones, which mattered because common categories dominate the score. Complement Naive Bayes alone was much weaker, likely too simple for the overlapping categories. An SGD classifier was close to the Linear SVM but slightly worse and less stable, and an SGD-SVC ensemble that used SGD only where it validated better gave a small validation gain but did not transfer to the leaderboard, appearing to overfit the split.

4.4. Feature Engineering and Rule-Based Improvements

Abbreviation expansion (for example dm, hta, irc, epoc) did not help, as many abbreviations are context-dependent and the character model already handled them reasonably. A short-literal exact lookup and a set of engineered surface features (length, word count, presence of digits or code-like patterns) added complexity without clear gains, since TF-IDF already captured most useful surface signal.

A broad context-flag model using manual indicators for obstetric, newborn, respiratory, infection, urinary, oncology, trauma, and procedure terms helped some examples but hurt others by over-pulling predictions. Restricting this to a few reliable patterns, a selective context override keyed on terms such as puerperio, puerperi, and fetal, became one of the strongest Phase-1 submissions, adding domain information without overcorrecting. Manual O/Z rescue rules improved validation but not the leaderboard, indicating overfitting to the local split, and were rejected.

5. Error Analysis and Data Quality

We analysed Linear SVM decision margins and found that low-confidence rows had lower validation accuracy, confirming the margin as a useful diagnostic, though confidence-based fallbacks did not improve the final score. A confusion matrix highlighted recurring problems with categories O and Z and with some numeric procedure categories.

A root-cause error table grouped mistakes into unseen literals, ambiguous literals, short abbreviations, code-like strings, and overpredicted categories. The largest sources were unseen and ambiguous literals. In particular, the same cleaned literal was sometimes associated with multiple categories, for example rubeola no inmune, hipotiroidismo part, obesidad morbida parto, and asma part.

We tried several data-quality remedies. Purity-weighted training down-weighted ambiguous literals but slightly worsened results, suggesting those rows still carried useful signal. High-purity-only training removed ambiguous examples entirely and also reduced performance, as too much realistic data was lost. Majority-label deduplication is the best text-only approximation for conflicting labels but cannot resolve cases that genuinely require missing clinical context. A two-stage classifier that first predicted numeric versus alphabetic and then classified within the group performed worse, because first-stage errors could not be recovered later.

6. ICD-9 to ICD-10 Mapping Experiments

We investigated whether the flagged ICD-9 rows could help with unseen literals. Hundreds of test literals appeared only in the flagged ICD-9 data and were missing from the ICD-10 training set, suggesting potential value if converted safely.

A first broad range mapping (for example ICD-9 240–279 to E, 390–459 to I, 460–519 to J) reduced the score, being too crude. We then used official General Equivalence Mapping (GEM) files, which were more principled but still imperfect, since ICD-9 to ICD-10 conversion is not always one-to-one; we kept only safer, consistent mappings. A GEM exact override that replaced predictions on matching ICD-9 literals slightly reduced performance, showing the mappings did not always align with the competition labels. Adding GEM-mapped ICD-9 rows as weak training data with reduced sample weight did not beat selective context either. The best use was a constrained GEM override on top of selective context, allowing only safer disease-like categories and excluding noisy ones such as S, T, and Y as well as risky obstetric contexts; this produced a small improvement, confirming that ICD-9 data helped only when applied very selectively.

Finally, we compared submissions row by row to find where methods disagreed. This revealed that several methods changed many predictions without improving the score, meaning they altered both correct and incorrect rows. This disagreement analysis informed the final constrained override and is best understood as targeted post-processing rather than a new model.

7. Phase 2: Transformer Augmentation and Probability Blend

Phase 1 plateaued near a public score of 0.572, so we added a neural component.

Clinical transformer. We fine-tuned PlanTL-GOB-ES/roberta-base-biomedical-clinical-es, a RoBERTa model pre-trained on Spanish biomedical and clinical text by the Barcelona Supercomputing Center, as a chapter classifier with a class-weighted loss to counter the imbalance. We also incorporated external ICD-coded clinical data from the CodiEsp corpus as additional training material.

Probability blend. Rather than overriding one model with another, we combined the classical ensemble and the transformer at the probability level:

$$P_{\text{final}} = \alpha P_{\text{transformer}} + (1 - \alpha) P_{\text{classical}}, \tag{1}$$

predicting the argmax of P_{final}. Because local validation did not reliably track the leaderboard, we tuned α by submitting a small set of candidate values and reading the public scores. The best value was approximately $\alpha = 0.24$, meaning the classical model anchors the prediction while the transformer contributes complementary signal.

Re-tuned regularisation. For the classical component inside the blend, the Linear SVM regularisation was re-tuned from the Phase-1 value of C = 0.5 to a slightly stronger setting of about C = 0.25, which combined best with the transformer probabilities. We also folded in light Spanish stopword removal and stemming, worth roughly +0.4 points on local validation.

8. Results

The final blended system reached a public leaderboard accuracy of 0.595, up from the 0.572 reached by the best Phase-1 classical configuration. The components were weaker individually: the classical ensemble scored 0.544 and the fine-tuned transformer around 0.50 in isolation. The fact that the blend exceeds both suggests that the transformer, although less accurate alone, contributes some correct predictions the classical model misses. The blend weight had a smooth effect on the leaderboard, peaking near $\alpha = 0.24$ and degrading gradually on either side.

For the model comparison in Table 1, each submitted system was trained on the full labelled training set instead of reserving a fixed validation split. The main comparison metric was therefore the public Kaggle leaderboard score. Local validation splits were used only as diagnostic tools during development, because their ranking of methods did not consistently match the leaderboard.

Table 1. Comparison of the main individual models and the final blended system.

System	Public score
Majority vote	0.560
SVD embedding	0.486
SVC plain	0.559
Logistic Regression	0.521
Classical ensemble alone	0.544
Best Phase-1 classical configuration	0.572
Fine-tuned clinical transformer alone	∼0.50
Final probability blend	0.595

Several Phase-2 experiments did not help. Replacing the Spanish biomedical model with a general multilingual one lowered the transformer's accuracy substantially, reinforcing the importance of domain pre-training. Adding the full reference description table, adding the CodiEsp corpus, and averaging multiple transformer runs each changed the blended score only negligibly. As in Phase 1, reweighting toward the numeric procedure categories raised procedure accuracy but lowered diagnosis accuracy by an equal amount, with no net gain.

9. Discussion

To understand why the score plateaued near 0.595, we returned to the error analysis. The largest single source of error was distinguishing diagnoses from procedures. In a two-stage experiment, if the diagnosis-versus-procedure label were known perfectly, overall accuracy would rise. However, predicting that label from the literal alone reached only about 75% accuracy, and that error rate cancelled most of the potential gain. This suggests that the main bottleneck was missing clinical context, not only insufficient model capacity.

The final blend is therefore best interpreted as a useful but limited gain. The classical model remained strong because TF-IDF character and word features match the surface form of these short literals very well. The transformer added complementary information, but it could not fully resolve cases where the literal itself did not contain enough evidence. This also explains why several more complex additions, such as extra corpora or broader rule systems, produced only small or unstable changes.

10. Conclusion

The best result was obtained by combining the classical ensemble with the fine-tuned clinical transformer at the probability level, reaching a public Kaggle score of 0.595. This improved over the best Phase-1 classical configuration and showed that the transformer was useful as a complementary signal, while the strongest overall system still depended on robust TF-IDF features and careful post-processing.

Overall, the project shows both the value and the limit of text-only modelling for this task. The remaining errors are largely tied to ambiguous or underspecified literals, especially diagnosis-versus-procedure cases. Further improvement would likely require additional clinical context or structured information, rather than only larger models or more aggressive tuning of the isolated text classifier.

References

[1] Scikit-learn developers. Scikit-learn: Machine Learning in Python. https://scikit-learn.org

[2] Barcelona Supercomputing Center. roberta-base-biomedical-clinical-es. https://huggingface.co/PlanTL-GOB-ES/roberta-base-biomedical-clinical-es

[3] A. Miranda-Escalada, A. Gonzalez-Agirre, J. Armengol-Estapé, M. Krallinger. Overview of automatic clinical coding: CodiEsp track of CLEF eHealth 2020. https://temu.bsc.es/codiesp/